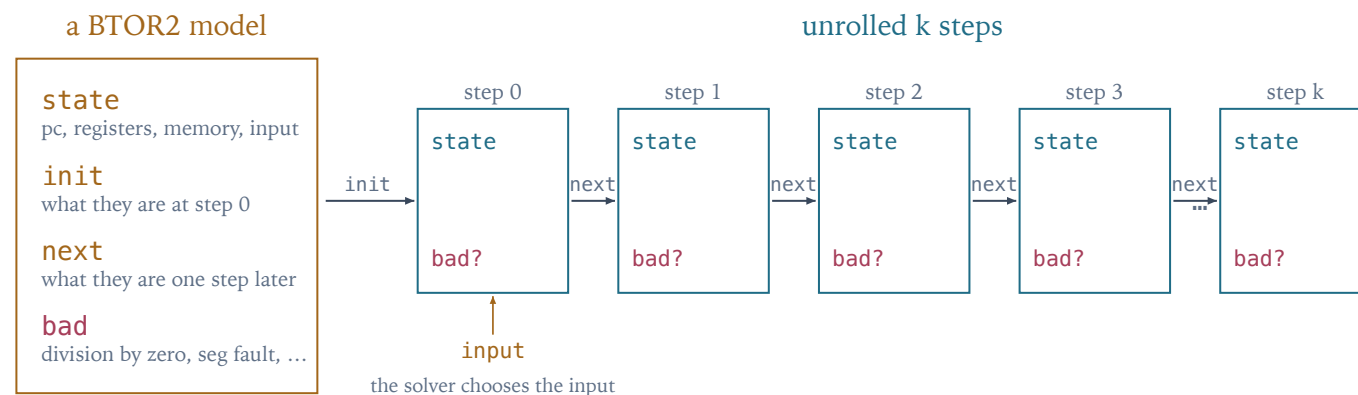


Verifying Systems Code

Rotor turns a program into a formula. Bitme asks a solver whether any input reaches a bad state within a bound.

[← SE 09 · Runtime Systems](#)

A machine state is a formula. A step is a formula. Reaching a bad state in k steps is a **SAT** question.



satisfiable $\Leftrightarrow$ some input reaches a bad state within k steps $\Rightarrow$ the satisfying assignment is the failing input
 unsatisfiable $\Leftrightarrow$ no input reaches a bad state within k steps $\Rightarrow$ a theorem, up to the bound k , and nothing beyond it

size of the unrolled formula: $O(k \times \text{size of the model}) \cdot \text{size of the model}$: linear in the size of the binary

a BTOR2 model: state, init, next, bad, unrolled k steps

Registers and memory are bit-vectors. Each of the fourteen instructions is a function from state to state, expressible in the theory of bit-vectors and arrays. A program's semantics, from the Meaning chapter, is exactly the transition relation of a BTOR2 model: *init*, *next*, and a list of *bad* states.

Unroll *next* k times, conjoin one bad state, hand the formula to an SMT solver. Satisfiable means an input exists that reaches the bad state at step k , and the satisfying assignment is that input. Unsatisfiable for all k up to a bound means no input reaches it within the bound: a theorem.

ROTOR

Rotor generates the model of a RISC-U program **with** its system calls.

```
$ ./rotor -c examples/symbolic/division-by-zero-3-35.c
... writing division-by-zero-3-35-rotorized.btor2
bad states, among others:
  core-0-division-by-zero
  core-0-fetch-seg-fault
  core-0-load-seg-fault
  core-0-store-seg-fault
  core-0-brk-seg-fault
  core-0-bad-exit-code
```

Rotor is selfie's model generator. It compiles the program, then emits the machine as a formula: registers, memory segments, the fourteen instructions, and the system calls as what the kernel does to the state. Input bytes are the free variables.

The bad states are systems properties: an address outside the mapped segments, a fetch from data, a store into code, a break that moves backwards, a non-zero exit. Memory safety, as formulas over the state the kernel manages.

```
$ tools/bitme.py -kmax 120 --use-bitwuzla examples/symbolic/division-by-zero-3-35-rotorized.btor2  
bitme bounded model checking: -kmin 0 -kmax 120  
0: initializing · 1: transitioning · ... · 75: transitioning  
76: core-0-division-by-zero  
76: vvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvvv  
76: 150 state 71 input-buffer ; uninitialized input buffer  
76: state150-76 = (store ... #b0 #b00110000)          ← byte 48, the character '0'  
89: core-0-division-by-zero ... #b00110010            ← byte 50, '2': the second division  
106: core-0-bad-exit-code ... #b00001100             ← byte 12: exit(0), the flagged exit
```

The same run on a segmentation fault looks identical, with a different bad state and an input that computes a bad address. That is the point for a systems engineer: the exceptions your kernel handles are the properties the model checker searches for, and the search is exhaustive up to the bound.

Absence, up to k . Nothing about $k+1$. And nothing about **threads**, yet.

VERSUS TESTING

A test visits one state in a space of 2^{35} bits per machine. A bounded run covers every input and every path to depth k . Presence versus absence; observation versus theorem.

VERSUS RICE

No tool decides whether a program segfaults. This one decides whether it does within k steps, which is a different, finite question. The bound is the approximation, chosen and named, like the timer and the TCB.

VERSUS CONCURRENCY

Rotor models one machine context. Interleavings, week 8's state space, are not in the formula. Modelling them is a research problem with real tools, and not this class's. The formal-methods assignment stays sequential, and says so.

ASSIGNMENTS

rotor-bounds and treiber-stack.

- 1 rotor-bounds. Take a small sequential routine of your systems code, your assembler's parser or your malloc, and write a C* driver that reads its input with read.
- 2 State the property in rotor's terms: which bad state. Generate the model; run bitme with a bound you justify.
- 3 Submit driver, property, bound, and the input the solver found or the bound up to which none exists, with a page on what the verdict establishes.
- 4 treiber-stack. `init_stack`, `push`, `pop` on a linked list with one shared top, using `lr` and `sc`, so the ABA problem cannot arise. Nodes from your thread-safe malloc.

GRADING ROTOR-BOUNDS

The grader runs rotor and bitme on your driver and compares verdict and bound; the invocation is in the assignment file. Two weeks for both. The stack is the harder code; the model is the harder thinking.

Listen, then read.

LISTEN

Bach · Canonic Variations on “Vom Himmel hoch” — Robert Costin, organ. One hymn, five canons at fixed intervals, each checked strictly against the rule while the tune runs through unchanged; the last inverts and augments at once. A property, a bound and a model, in 1747.

READ

Decision Procedures: An Algorithmic Point of View — Kroening and Strichman: the textbook on SMT, bit-vectors, arrays and bit-blasting, written for exactly the purpose of checking programs. And, more technical, Symbolic Model Checking without BDDs — Biere, Cimatti, Clarke and Zhu, 1999: bounded model checking, the k that bitme runs, in the paper that invented it.